

Multi-Currency Payment

Buzzvel 2026 Dev Team Test

Lisbon, 8 June 2026

Version 1.0

A note on AI

We work with AI tools every day at Buzzvel, and we encourage you to do the same during this test. Feel free to use any AI assistant (ChatGPT, Claude, Copilot, Cursor, or others) to help you along the way.

That said, we also know that when AI is available, there's no excuse for a poor UI. We expect the frontend to look clean and polished; if anything, having AI at your disposal raises the bar. We'll be paying attention to design quality, not just functionality.

What we're evaluating is your ability to use these tools effectively, make good decisions, and produce something you're proud of.

Story

In this test, you'll build a Laravel 12 application that manages multi-currency payment requests across a company with employees in different countries. Your goal is to create a functional payment request service where authenticated users can submit payments in their local currency, fetch real-time exchange rates, and route requests for approval by the finance team.

You must implement a complete authentication system, payment request management, exchange rate integration, and comprehensive testing. This project will test your proficiency with Laravel, PHP 8.2+, and modern backend development practices in a real-world scenario reflecting Buzzvel's technical environment.

Requirements

- **Authentication:** Implement authentication using Laravel Passport or another preferred mechanism (Registration, Login, Logout)
- **Payment Requests:** Create a complete system to manage payment requests:
 - **Create:** Submit a payment request in local currency (exchange rate fetched automatically)
 - **Read:** Fetch payment request details and list with status filter
 - **Update:** Approve or reject a pending request (finance role)
- The exchange rate must be stored at the time of creation and remain immutable.
 - **Exchange Rate Integration:** Integrate with a free public exchange rate API (e.g., <https://api.exchangerate-api.com>) to

- Fetch the real-time EUR → local currency rate on request creation
 - Store the rate, source, and timestamp alongside the payment record
 - Return the converted EUR amount in the API response.
- Implement the scheduled task of automatically expiring payment requests that remain pending for more than 48 hours
- Implement appropriate validation for all input data (required fields, currency codes, amounts).

Documentation

- Provide documentation for each API endpoint, including request method, URL, request/response parameters, and example responses.

Unit Testing

- Write unit tests for at least the critical functionalities of the API.

Additional Guidelines

- Use Laravel best practices and conventions.
- Follow RESTful API design principles.
- Handle errors gracefully and return meaningful error messages.
- Use Git for version control and share your code repository with us upon completion.
- Provide clear instructions on how to set up and run your project locally.
- Seeders with at least **5 employees** across different countries and currencies.

Evaluation Criteria

- Code structure and organization.
- REST API design and implementation.
- Error handling and validation.
- Authentication implementation.
- Exchange rate integration correctness.
- Testing approach and coverage.
- Overall code quality and readability.

Submission

A README is required. If we don't have a README, this test will be ignored.

A video or URL showing the project is working. You can upload and share with a public link, and send us the link.

Please submit your solution within the specified time frame. Include your project code, setup instructions, and documentation in this form:

<https://forms.clickup.com/6647387/f/6avjv-18455/PLUYAZ40HA3XTQOEFW>

Timeline

- **5 days after receiving the test:** Final day to deliver
- **10–15 days after:** Code review
- **After step 2:** Interview
- **After step 3:** Final decision

